Lab W10: The Guild Commission Board, Part 1

CIS 3300, Intermediate Object-Oriented Programming

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

---

1 The Story

Townsfolk who need help post paid jobs, called commissions, at the Coppertop Guild Hall. The guild clerk records every posting in a board file. Each commission has an ID, a title, the client who posted it, and the lowest guild rank allowed to take it, plus the details that matter for its kind of job. The guild takes four kinds of commissions, and each pays by its own rule:

- **Hunts** pay 40 gold per threat level for each creature.
- **Escorts** pay 15 gold per traveler per day, and never less than 60 gold.
- **Deliveries** pay 2 gold per mile. A rush delivery pays half again as much.
- **Retrievals** pay a finder's fee of 30 gold plus 10 percent of the item's value.

The guild keeps 10 percent of every payout, and the member who completes the commission receives the rest. The guildmaster wants a report on the current board. This is the first of three parts. Labs W11 and W14 build on the code you write here.

2 What You Are Building

- **Classes that represent the commissions** on the board. Your classes keep everything the board file records about each commission, even information this report does not use yet.
- **A program with a `main` method** that reads the board file the way we did in lecture and writes a report to a text file named `board_report.txt`, the way we did in lecture.

The report includes:

- Every commission on the board, with its details, its payout, and the share the member receives.
- The total payout for each kind of commission, the total for the whole board, and the guild's total share.

- The Danger Board: every hunt with a threat level of 7 or higher.

A sample report for the `board` file, `sample_board_report.txt`, shows the layout your report must follow. Match it closely: the same sections, in the same order, with the same information. Exact spacing may vary within reason, but your report must stay clean and aligned for every board file, not just `board`. The other files have more commissions and longer titles and names, so plan your columns with that in mind.

You may add any other fields, methods, or classes you find useful.

3 Many Decisions Are Yours

This assignment deliberately does not tell you which classes to write, what to name them, or how they relate to each other. You decide, and you will need to be able to explain why. This does not mean that "anything goes" for points.

There is more than one good answer, and your design does not need to match a classmate's. What matters is that your decisions are deliberate, consistent, and match what your code actually does.

Your decisions and your code must be consistent with what we have covered through Week 10, in both the lectures and the assigned book chapters; this assignment is about applying that knowledge. Your design must use inheritance and polymorphism in a meaningful way. A design that avoids them, or uses them only on the surface, does not meet the expectations for this assignment.

4 Design Vocabulary

These terms describe the qualities of a good object-oriented design. The short definitions below are only a starting point. You are expected to find out more about each one on your own, using the textbook and other reliable sources.

- **Encapsulation:** a class keeps its data private and controls every way that data can change.
- **Cohesion:** each class has one clear job, and everything in it belongs to that job. High cohesion is the goal.
- **Coupling:** how much classes depend on each other's details. Low coupling is the goal.
- **Reusability:** shared code is written once and reused instead of copied.
- **Extensibility:** new features can be added mostly by adding new code, not by changing code that already works.
- **Substitutability:** an object of a subclass can be used anywhere its superclass is expected, and it behaves sensibly there.

5 Explaining Your Decisions

You explain your design in comments in your code, written in your own words. Selected students may be asked to walk through these comments in a personal review, as described in the syllabus.

- **Your decisions.** Explain every major design decision you made, in a comment placed right next to the code where you made it, not gathered at the top of a file. Each explanation can be

short, but it must be clear enough that a reader understands what you decided and why. Start each of these comments with `DECISION:`.

- **One design term.** Choose one term from the Design Vocabulary. Write a comment beside the code that best shows that term, starting with `DESIGN TERM:` and the term's name. In it, describe how your design reflects the term, a weaker design and what makes it weaker, and a better design and what it would cost.

For this assignment, you do not need the full commenting described in the syllabus's Code Commenting Requirements. Each file still needs the general comment at the top (a description of the class, the assignment title, a version number, and your name). Beyond that, put your effort into comments that explain your design, not into documenting every method and field.

5.1 Examples

These examples come from a different program, a coffee shop that sells several kinds of drinks, so they show the form your comments should take without describing your design. Yours should be about as short and direct.

```
// DECISION: Drink is abstract. Every order is a specific kind of drink,
// and a generic drink has no price rule of its own.
public abstract class Drink {
    ...
    // DECISION: price() is abstract so every kind of drink must supply
    // its own rule. The compiler rejects a new drink that forgets one.
    public abstract double price();
}
```

```
// DESIGN TERM: Extensibility
// My design: each drink overrides price(). Adding hot chocolate means one
//   new class; the receipt loop below does not change.
// Weaker: one Drink class with a type field and an if chain in price().
//   A new drink means editing that method, and a missed branch charges
//   the wrong price without any error.
// Better, at a cost: read prices from a file instead of constants, so
//   prices change without changing code. Cost: more code to read and
//   check the file.
for (Drink d : order) {
    total += d.price();
}
```

6 Project Setup

1. Create a new Java project named `labw10`.
2. Set up anything the project needs to read the board file, the same way we did in lecture.
3. Put your classes in packages: the classes that represent commissions in a package named `guild`, and your program with `main` in a package named `app`.
4. Create a `files` folder in the project's root folder. Place the `board` file and the sample report, `sample_board_report.txt`, in it, and write your report to that same folder.

7 Additional Sample Data

Two more board files, `board2` and `board3`, each larger than the last, are provided for testing. Place them in your `files` folder. Your classes should work with any of them without changes. To test with one of them, change the file name your program reads and run it again. The program you submit should read `board`.

8 Requirements Checklist

- Your design uses inheritance and polymorphism in a meaningful way, along with encapsulation and the other principles of good class design we discussed in class.
- Each kind of commission is paid according to the guild's rules.
- Your program reads the board file the way we did in lecture.
- Your program writes a complete report to a text file the way we did in lecture, closely following the layout of the sample report, and it stays clean and aligned for every board file.
- Your `DECISION:` comments explain every major design decision next to the code that carries it out, and your code matches what they say.
- Your `DESIGN TERM:` comment describes your design, a weaker design, and a better design with its cost.
- Your code meets the course's standards for code quality, and every file has the required header comment.

9 Grading

Items in Tables 1 and 2 are worth 1 point each and are either met or not met. Items in Table 3 are worth a range of points, with partial credit based on how well your work meets the standards we set in lecture. Because there is more than one good design for this assignment, grading of the Table 3 items includes some subjectivity.

Table 1: Submission and setup (1 point each)

Item	Points
Submitted as a reasonable attempt that compiles and runs in Eclipse without errors	1
Project laid out as described in Project Setup (project name, packages, and `files` folder)	1

Table 2: Code quality (1 point each)

Item	Points
Consistent naming conventions used throughout	1
Consistent formatting and indentation throughout	1
No unrequested extra output	1
Each file has the required header comment	1
Loop and method logic reads clearly from top to bottom	1

Table 3: Design and implementation (partial credit possible)

Item	Points
The commission classes meet expectations	12
Payouts and shares are computed correctly	5
The board file is read using the approach from lecture	5
The report is complete and written using the approach from lecture	5
Design decisions are explained in the code, and the code matches the explanations	6
One design term is discussed, with a weaker and a better design	6

Total: 46 points.

10 Submission

Submit through your course GitHub repository, per the syllabus's submission guidelines. Include your `src` folder with the `guild` and `app` packages, and the `files` folder with the board files. It is fine either way whether your report file is included, since your program creates it every time it runs.